HTTP Guide: Wireshark Analysis of HTTP Traffic

What is HTTP?

HTTP (HyperText Transfer Protocol) is the foundation of the World Wide Web. It is a request-response protocol that allows clients (typically web browsers) to communicate with servers to fetch web pages, submit forms, upload files, and interact with web applications.

- **Version matters**: Most modern traffic is **HTTP/1.1** or **HTTP/2** (often encrypted as HTTPS). Older or debugging traffic may still appear as plain HTTP.
- **Stateless by design**: Each request is independent (though cookies and sessions maintain state).

Typical Ports

- **Port 80** — Standard unencrypted HTTP
- **Port 8080** — Common alternative (especially dev servers, proxies)
- **Port 443** — HTTPS (HTTP over TLS) — the packets you see will be encrypted

In Thaghrah challenges you will mostly work with **plain HTTP** captures so you can see the full conversation.

Why Analysts Care About HTTP

- It carries the majority of user-visible web activity.
- Sensitive data (credentials, session cookies, personal information) is often sent in cleartext on HTTP.
- Misconfigurations, injection attacks, information leaks, and malware callbacks frequently appear in HTTP traffic.
- Understanding HTTP helps you spot phishing sites, command-and-control channels, data exfiltration, and broken access control.

Wireshark Fundamentals for HTTP Analysis

1. Basic Filtering

Bash

```
http                # All HTTP traffic
http.request        # Only requests (GET, POST, etc.)
http.response       # Only responses
http contains "password"   # Search inside HTTP payload
http.host == "example.com"
http.response.code == 200
http.response.code >= 400   # Client or server errors
```

2. Following the Conversation

- Right-click any HTTP packet → **Follow** → **TCP Stream** (or HTTP Stream in newer Wireshark).
- This reconstructs the full request/response in a clean view.

3. Key Fields to Examine

Request Line & Headers

- GET /path HTTP/1.1
- POST, PUT, DELETE, HEAD, etc.
- Host: — which server is being contacted
- User-Agent: — client browser / tool fingerprint
- Cookie: — session identifiers (very high value for attackers)
- Authorization: — Basic auth (base64, easily decoded), Bearer tokens
- Referer: — where the user came from
- Content-Type: and Content-Length:

Response Line & Headers

- Status codes:
    - **2xx** — Success
    - **3xx** — Redirects (watch for open redirects)
    - **4xx** — Client errors
    - **5xx** — Server errors
- Set-Cookie: — Server setting or updating session cookies
- Location: — Redirect target
- Server: — Web server software and version (useful for fingerprinting)

Body / Payload

- Form data (application/x-www-form-urlencoded or multipart/form-data)
- JSON responses (application/json)
- HTML, images, scripts, etc.

Common Analysis Scenarios in Thaghrah Challenges

1. **Credential Harvesting**
    - Look for POST requests containing username, password, email, etc.
    - Check if they are sent over plain HTTP (very bad!).
2. **Session Hijacking**
    - Extract Cookie: values from requests.
    - See if cookies lack Secure, HttpOnly, or SameSite flags.
3. **Directory Traversal / Sensitive Files**
    - Requests like GET /../../etc/passwd or GET /.git/
4. **Information Leakage**
    - Error pages revealing stack traces, database info, or internal paths.
    - Verbose Server: headers.
5. **Redirect Analysis**
    - Malicious or open redirects (Location: http://evil.com).

6. **File Upload / Download**
    - Look at Content-Disposition in responses for downloaded files.
    - Multipart POST for uploads.

Practical Workflow in Wireshark

1. Open the .pcap file.
2. Apply filter http and press Enter.
3. Look at the **Protocol** column — everything should show HTTP.
4. Sort by **Time** or **No.** to follow chronological flow.
5. Use **Statistics** → **HTTP** → **Requests** or **Load Distribution** for overview.
6. Right-click interesting packets → **Follow** → **TCP Stream**.
7. Export objects if needed: **File** → **Export Objects** → **HTTP** (great for pulling images, scripts, or files from the capture).

Security Red Flags (Things You Should Spot)

- Login forms over HTTP instead of HTTPS.
- Session cookies transmitted over HTTP.
- Passwords or API keys visible in URLs (query parameters).
- Unusual User-Agent strings (possible automation / malware).
- Large POST bodies that look like encoded commands.
- Frequent 404s or 403s that might indicate scanning.

Example Challenge Goals You Will Encounter

- Extract the administrator's password from a login POST.
- Find the session cookie and use it to impersonate a user.
- Identify the flag hidden in a JSON response.
- Determine what file was downloaded by a user.
- Reconstruct a full user journey across multiple HTTP requests.

Next Steps for You (the student):

- Download the HTTP challenge PCAP.
- Apply the filters above.
- Follow the streams.
- Look for the flag (usually a string like FLAG={...} hidden in a header, body, cookie, or URL).